



Departamento de Informática e Matemática Aplicada – DIMAp

Spring Cloud Gateway

Prof. Nélio Cacho

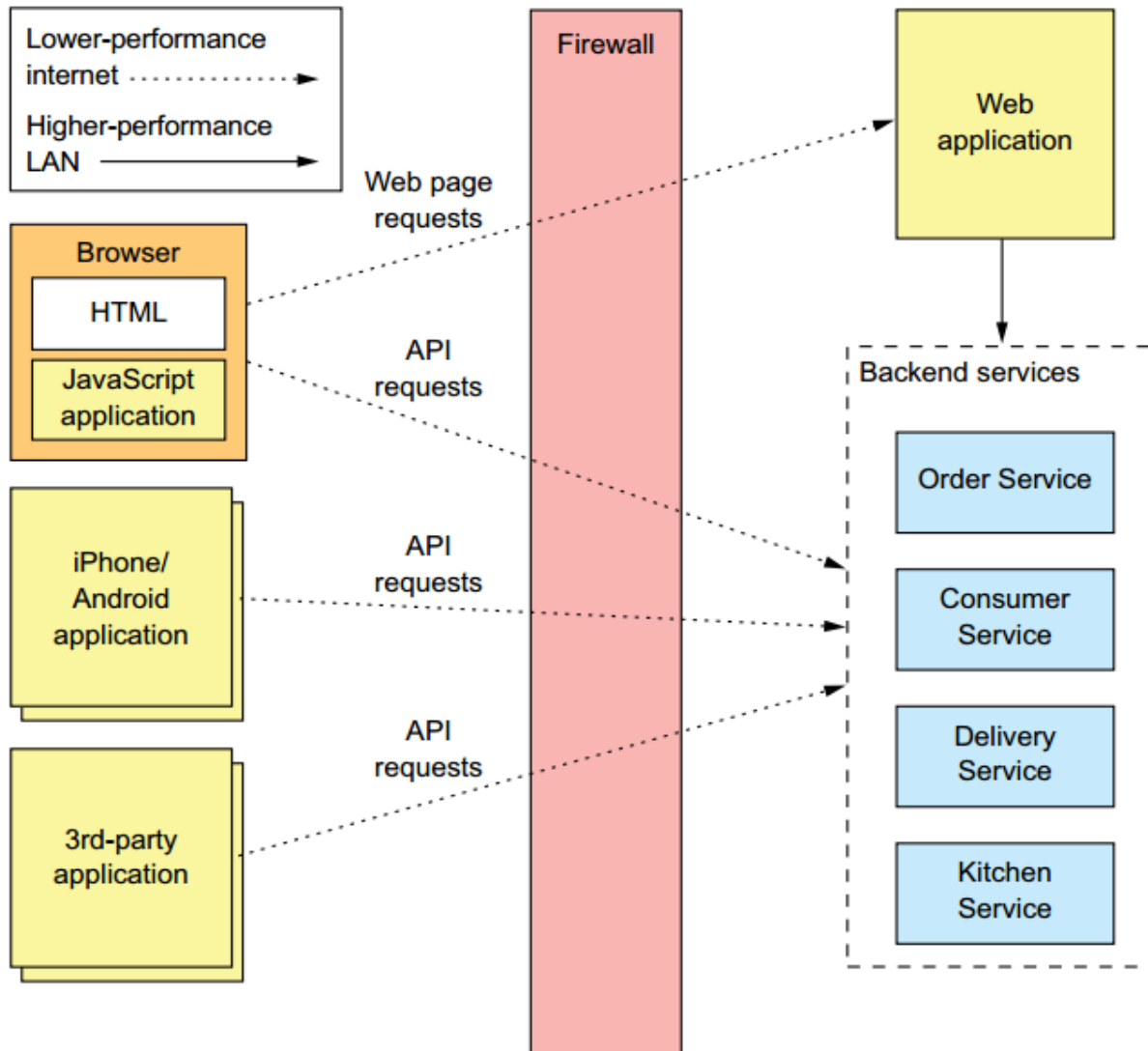
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



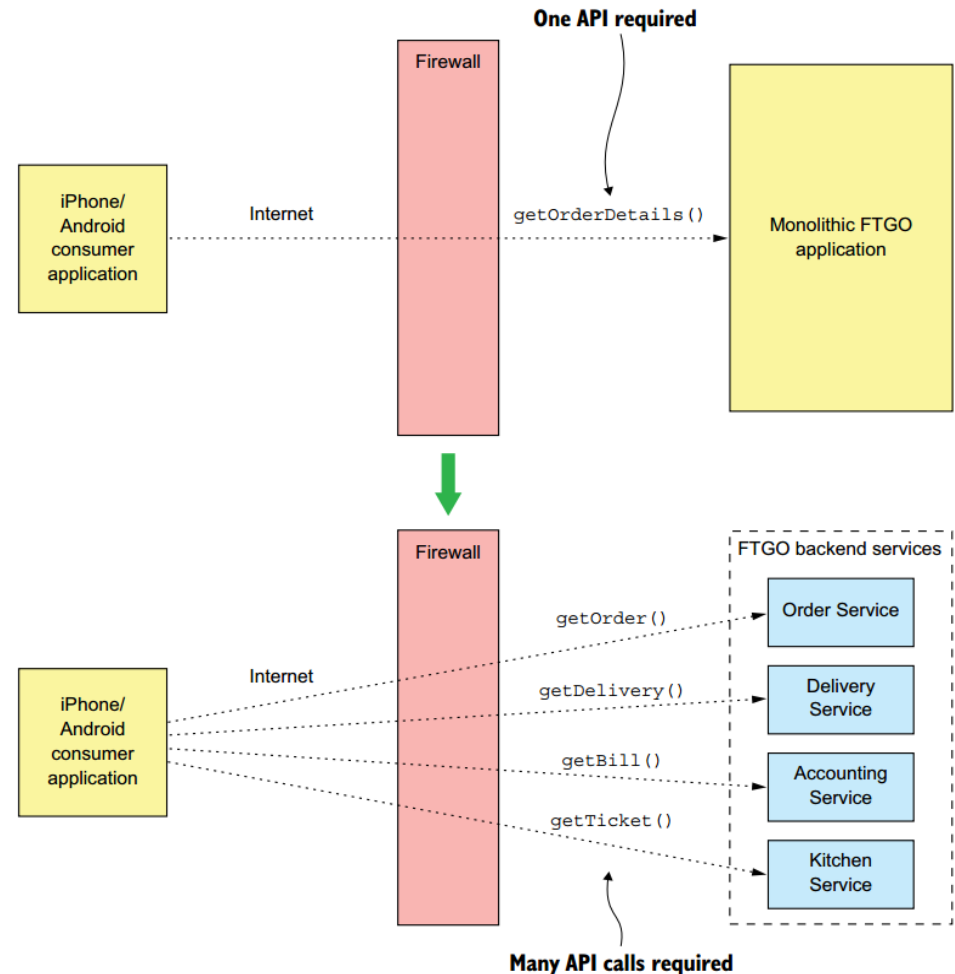
- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

Problemas de Projeto de API



Problemas de Projeto de API

- As APIs de serviço de granularidade fina exigem que os clientes façam várias solicitações para obter os dados de que precisam, o que é ineficiente e pode resultar em uma experiência ruim para o usuário.



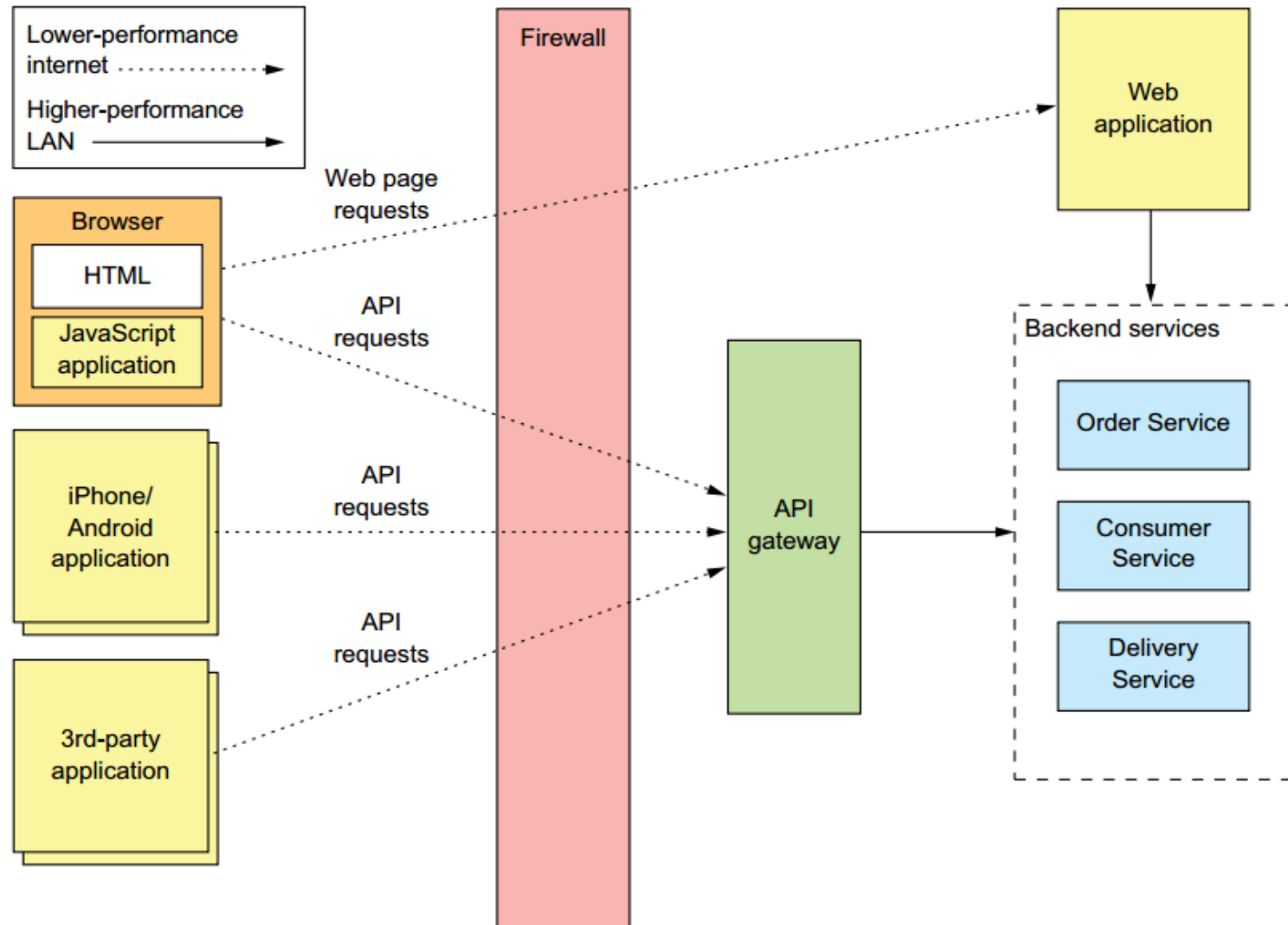
Problemas de Projeto de API

- A falta de encapsulamento causada pelo conhecimento dos clientes sobre cada serviço e sua API torna difícil a alteração da arquitetura e das APIs.
- Os serviços podem usar mecanismos de IPC que não são convenientes ou práticos para os clientes usarem, especialmente aqueles fora do firewall.

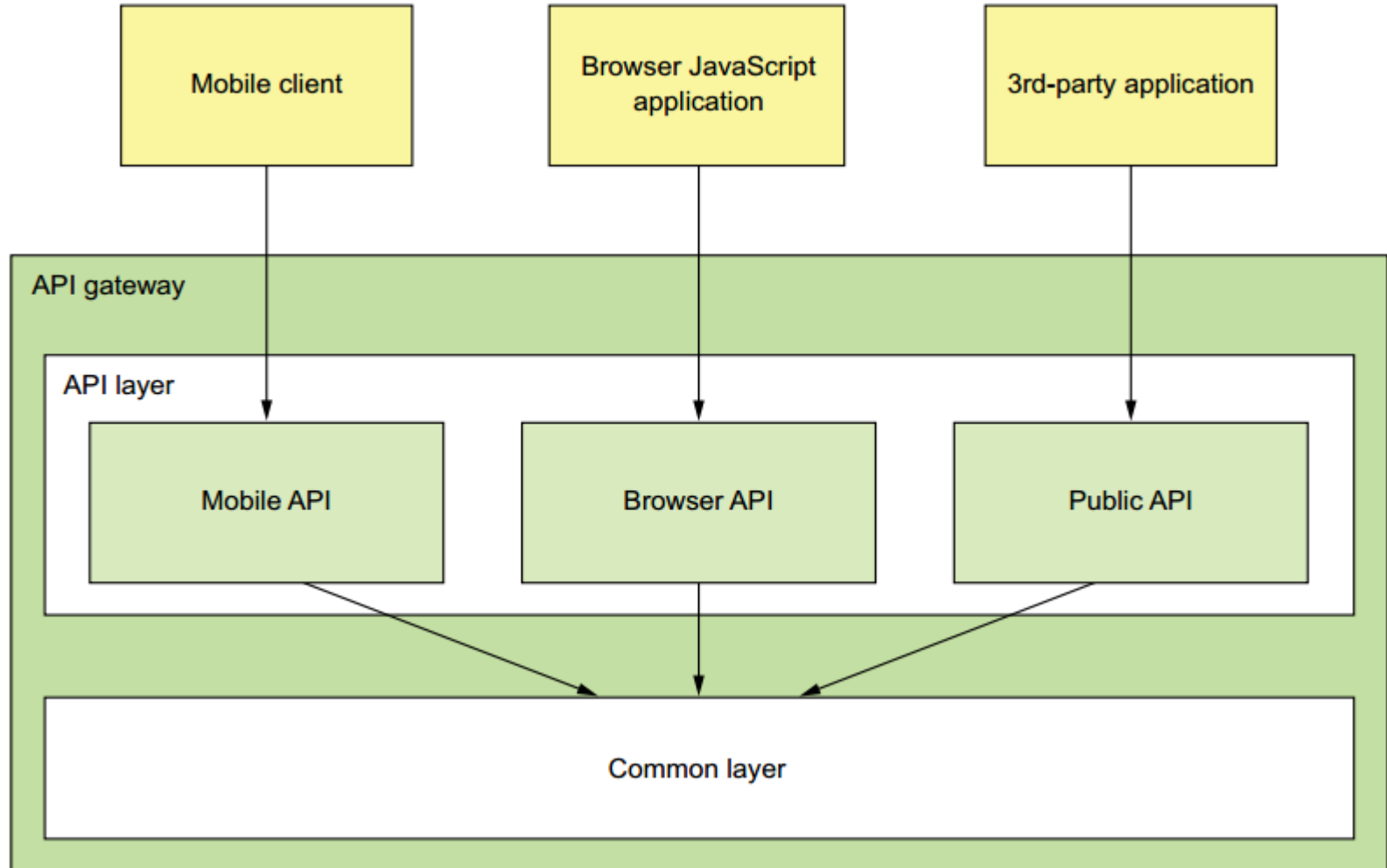
O Padrão de Projeto: API Gateway

- Um API gateway é um serviço que é o ponto de entrada para a aplicação a partir do mundo externo.
- Ele é responsável pelo roteamento de solicitações, composição de APIs e outras funções, como autenticação.

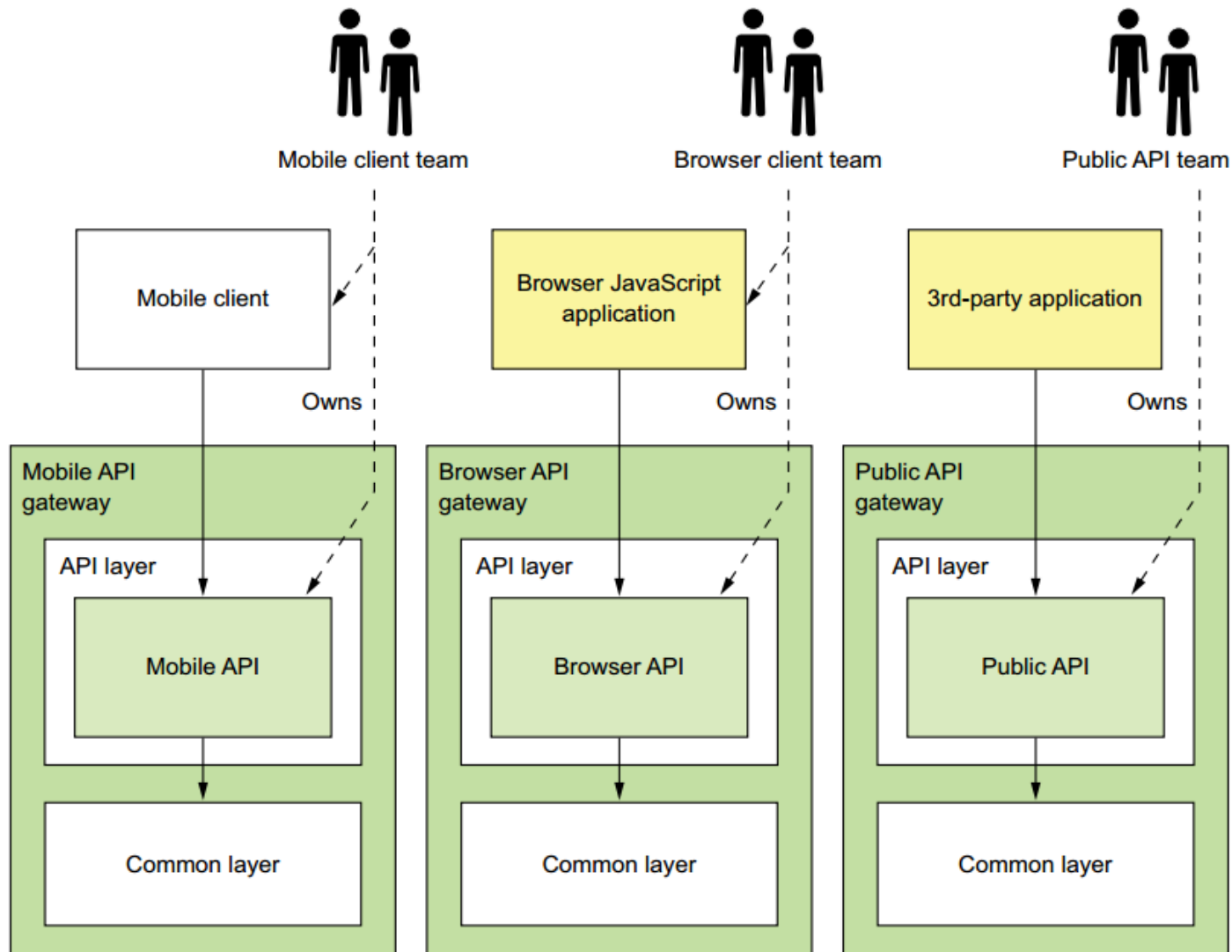
O Padrão de Projeto: API Gateway



Arquitetura do API Gateway



Backends for Frontends Pattern



Benefícios do API Gateway

- Um dos principais benefícios de usar um API gateway é que ele encapsula a estrutura interna da aplicação.
- Em vez de precisar invocar serviços específicos, os clientes se comunicam com o gateway.
- O API gateway fornece a cada cliente uma API específica do cliente, o que reduz o número de idas e voltas entre o cliente e a aplicação.
- Isso também simplifica o código do cliente.

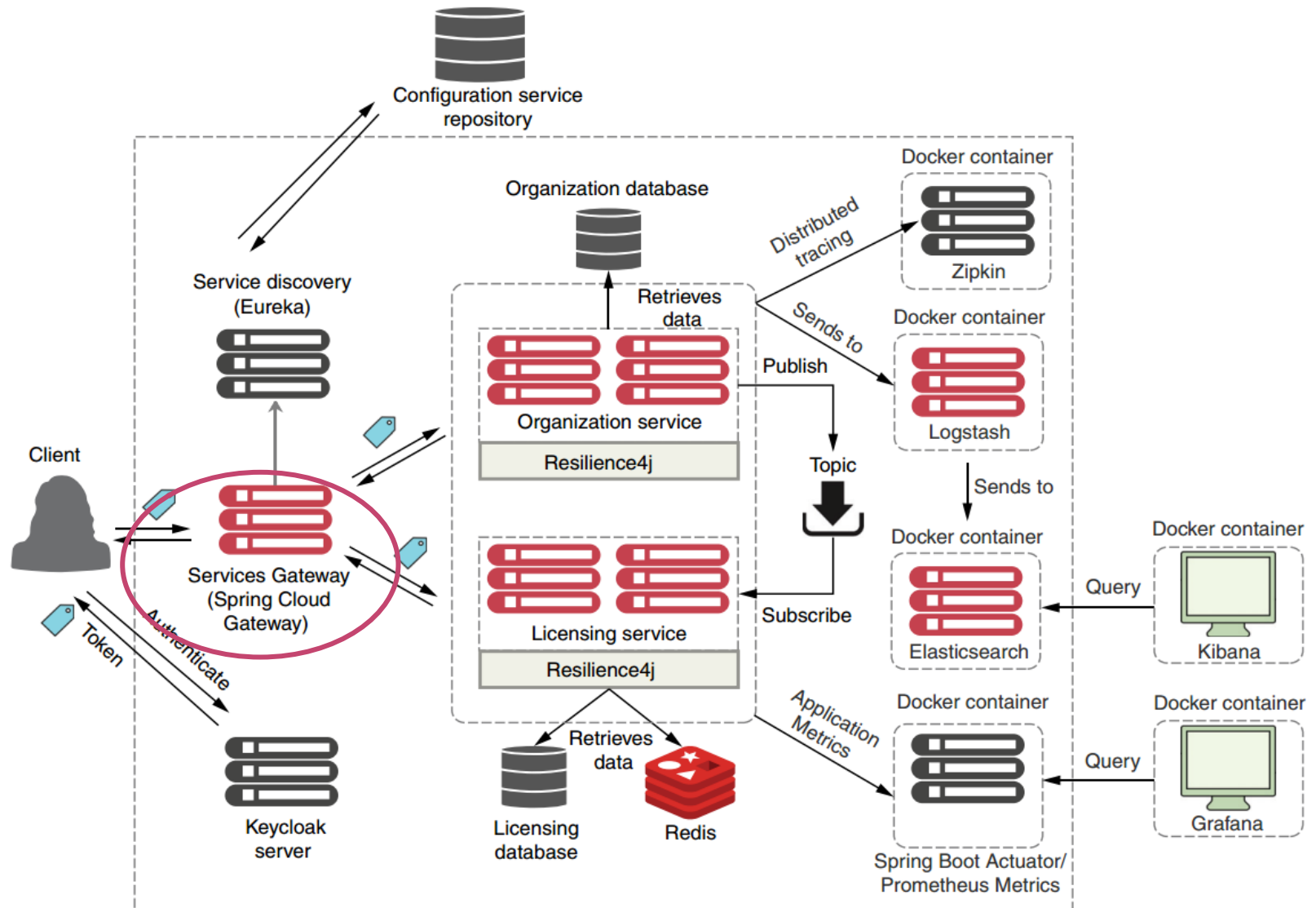
Limitações do API Gateway

- É mais um componente altamente disponível que precisa ser desenvolvido, implantado e gerenciado.
- Há também o risco de o API gateway se tornar um gargalo de desenvolvimento.
- Os desenvolvedores precisam atualizar o API gateway para expor a API de seus serviços.
- É importante que o processo de atualização do API gateway seja o mais leve possível. Caso contrário, os desenvolvedores serão obrigados a esperar na fila para atualizar o gateway.
- Apesar dessas desvantagens, no entanto, para a maioria das aplicações do mundo real, faz sentido usar um API gateway.
- Se necessário, você pode usar o padrão "Backends for Frontends" para permitir que as equipes desenvolvam e implantem suas APIs de forma independente.

Implementando um API Gateway

- Existem vários API gateways disponíveis, como:
 - AWS API Gateway
 - AWS Application Load Balancer
 - Kong (<https://konghq.com/>)
 - Traefik (<https://github.com/traefik/traefik>)
- Essas são algumas das opções populares para implementação de um API gateway em suas aplicações. Cada uma delas possui recursos e funcionalidades específicas, por isso é importante avaliar as necessidades do seu projeto antes de escolher a solução mais adequada.

Microservice patterns: The Big Picture



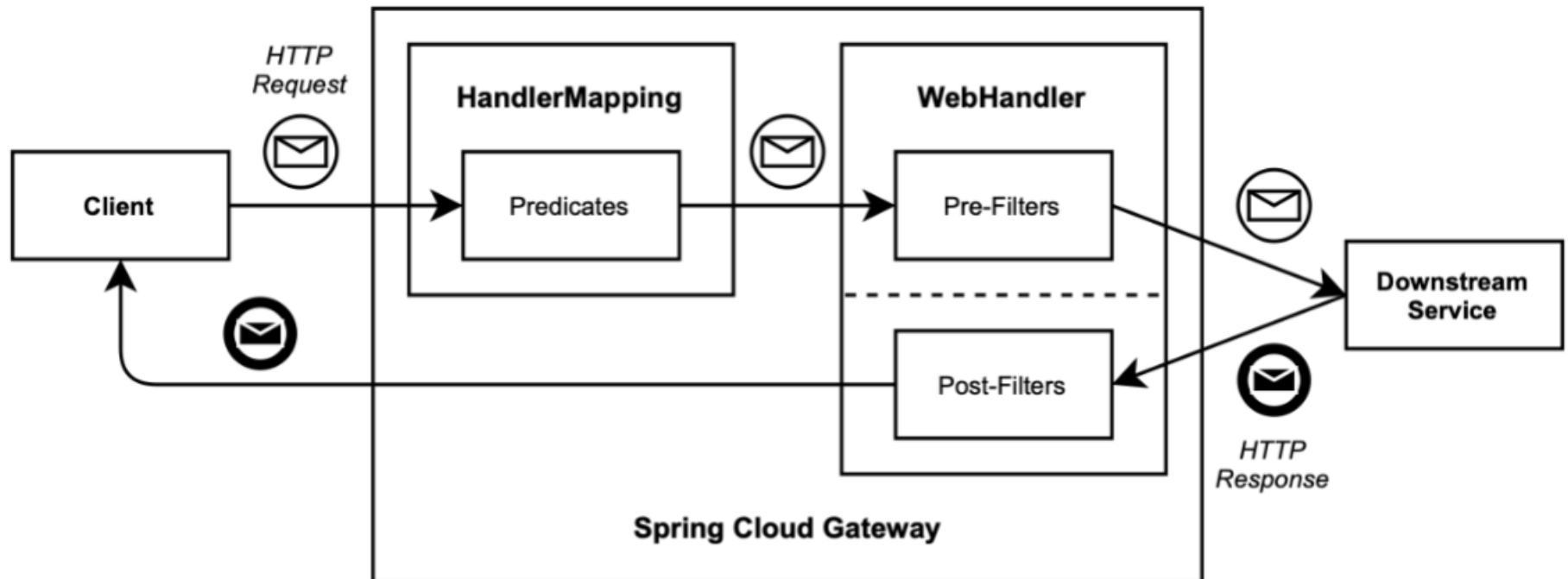
Spring Cloud Gateway

- O Spring Cloud Gateway é uma implementação de API gateway construída sobre o framework Spring 5, Project Reactor e Spring Boot 2.0.
- Esse gateway é não bloqueante. Aplicações não bloqueantes são escritas de tal forma que as threads principais nunca são bloqueadas.
- Em vez disso, essas threads estão sempre disponíveis para atender solicitações e processá-las de forma assíncrona em segundo plano para retornar uma resposta assim que o processamento for concluído.

Spring Cloud Gateway

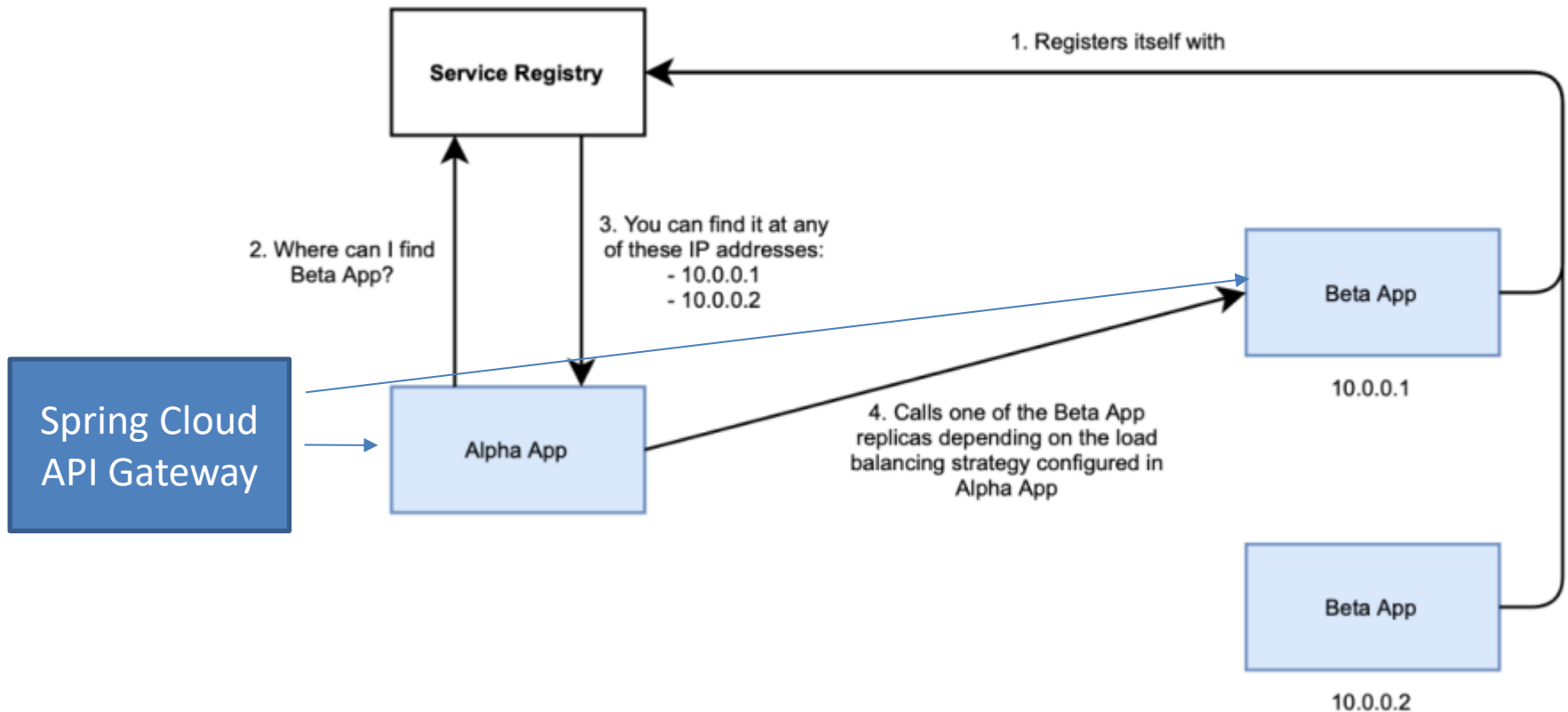
- O Spring Cloud Gateway fornece uma maneira simples e abrangente de fazer o seguinte:
 - Roteamento de solicitações para serviços de backend.
 - Implementação de manipuladores de solicitação que realizam composição de APIs.
 - Construção de filtros que podem inspecionar e agir nas solicitações e respostas que passam pelo gateway.
 - Construção de predicados, que são objetos que permitem verificar se as solicitações atendem a um conjunto de condições antes de executar ou processar uma solicitação.
- Com o Spring Cloud Gateway, é possível configurar e personalizar esses recursos de forma flexível para atender às necessidades específicas de roteamento, composição e manipulação de solicitações e respostas em sua aplicação.

Spring Cloud Gateway

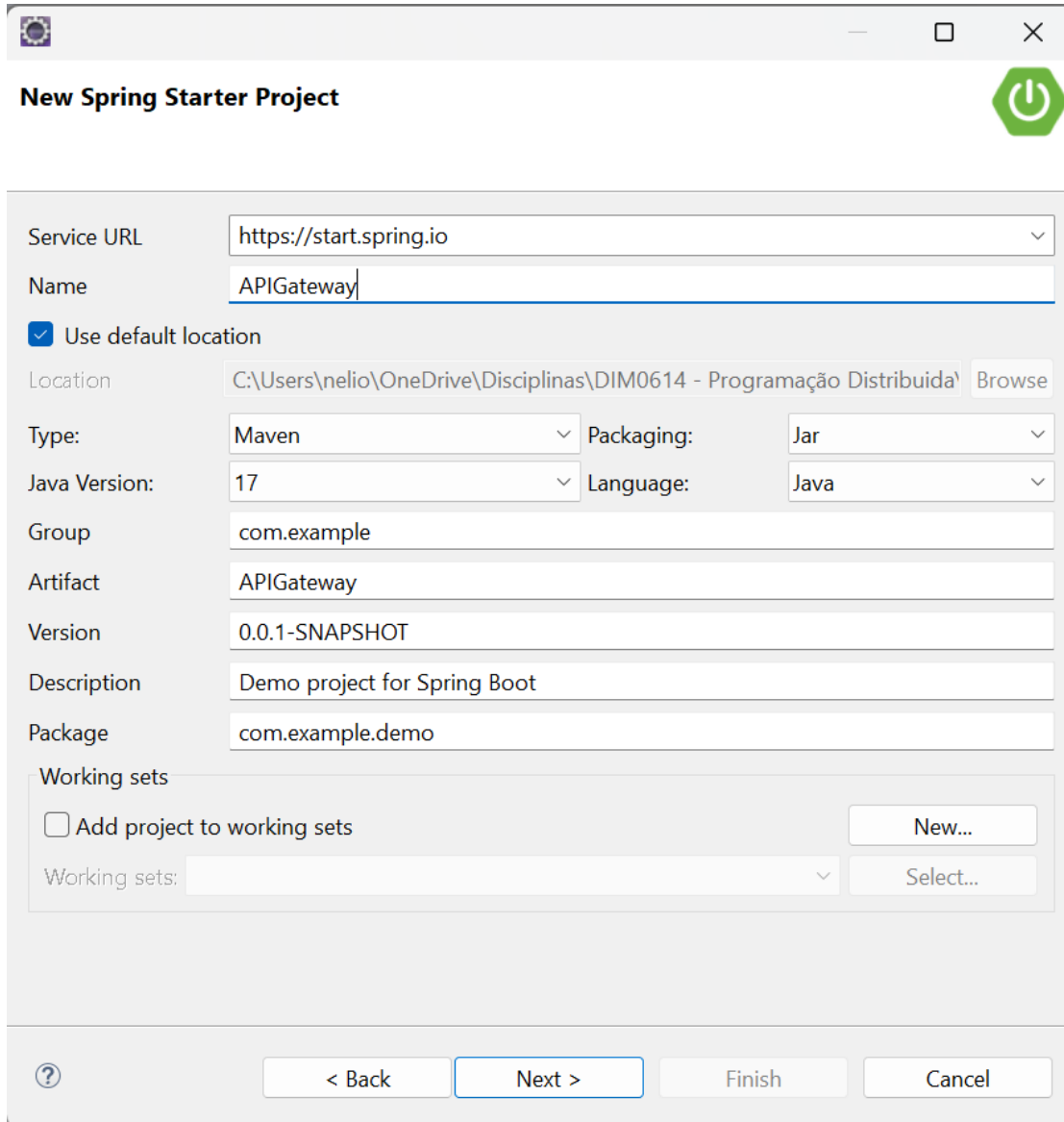


Exemplo de Uso do API Gateway

Client-side service discovery and load balancing - Model



Criando um API Gateway



The image shows a 'New Spring Starter Project' dialog box. At the top, there's a title bar with a Spring logo icon, and a green power button icon on the right. The main area contains several input fields and checkboxes. The 'Service URL' is set to 'https://start.spring.io'. The 'Name' field contains 'APIGateway'. The 'Use default location' checkbox is checked. The 'Location' field shows a file path: 'C:\Users\nelio\OneDrive\Disciplinas\DIM0614 - Programação Distribuida'. Below this, there are dropdown menus for 'Type' (Maven), 'Packaging' (Jar), 'Java Version' (17), and 'Language' (Java). Text fields for 'Group' (com.example), 'Artifact' (APIGateway), 'Version' (0.0.1-SNAPSHOT), 'Description' (Demo project for Spring Boot), and 'Package' (com.example.demo) are also present. At the bottom, there's a 'Working sets' section with an unchecked checkbox 'Add project to working sets', a 'New...' button, a 'Working sets:' dropdown, and a 'Select...' button. The very bottom has a navigation bar with a help icon, '< Back', 'Next >', 'Finish', and 'Cancel' buttons.

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

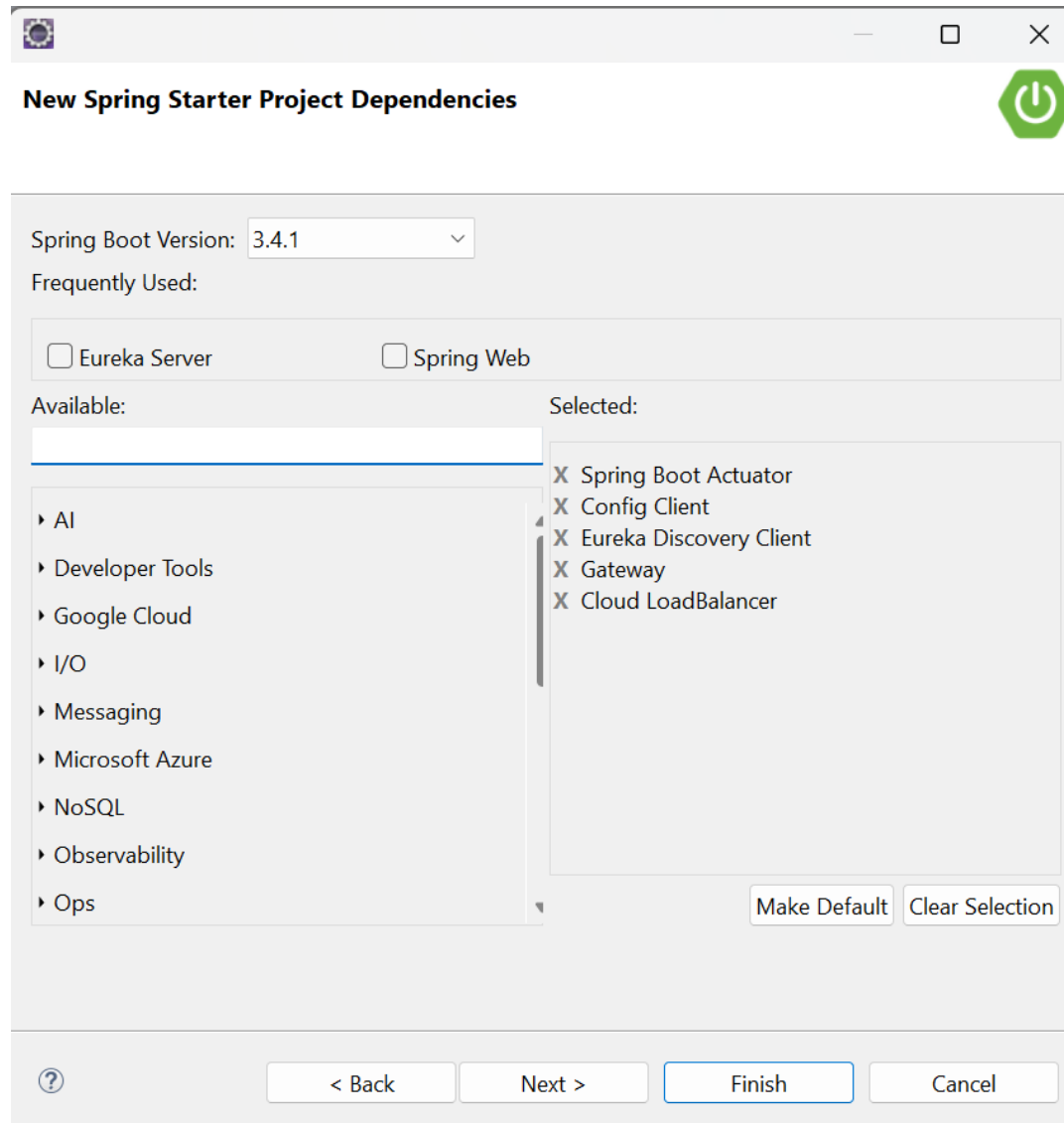
Package:

Working sets

☐ Add project to working sets

Working sets:

Criando um API Gateway



New Spring Starter Project Dependencies

Spring Boot Version: 3.4.1

Frequently Used:

☐ Eureka Server ☐ Spring Web

Available:

- ▶ AI
- ▶ Developer Tools
- ▶ Google Cloud
- ▶ I/O
- ▶ Messaging
- ▶ Microsoft Azure
- ▶ NoSQL
- ▶ Observability
- ▶ Ops

Selected:

- X Spring Boot Actuator
- X Config Client
- X Eureka Discovery Client
- X Gateway
- X Cloud LoadBalancer

Make Default Clear Selection

? < Back Next > Finish Cancel

Criando um API Gateway

```
@SpringBootApplication
@EnableEurekaClient
public class ServiceGatewayApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(ServiceGatewayApiApplication.class, args);
    }

}
```

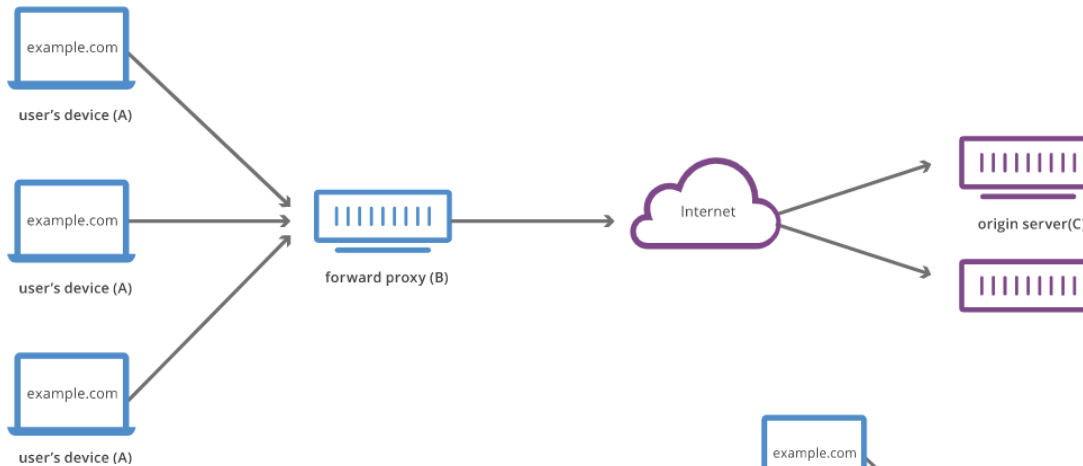
Criando um API Gateway

```
1  spring:
2    application:
3      name: ServiceGatewayAPI
4    cloud:
5      loadbalancer:
6        ribbon:
7          enabled: false
8
9  server:
10    port: 8080
11
12  eureka:
13    instance:
14      prefer-ip-address: true
15    client:
16      registerWithEureka: true
17      fetchRegistry: true
18      serviceUrl:
19        defaultZone: http://localhost:8761/eureka/
20
```

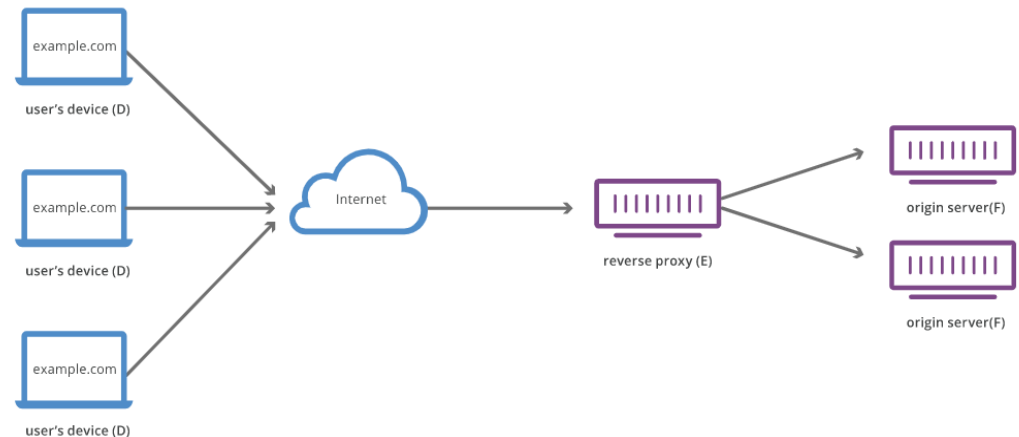
Configurando Rotas no API Gateway

- Forward Proxy vs Reverse Proxy

Forward Proxy Flow



Reverse Proxy Flow



Configurando o Spring Cloud Gateway

- **O Spring Cloud Gateway é um proxy reverso.**
- Um proxy reverso é um servidor intermediário que fica entre o cliente que está tentando acessar um recurso e o próprio recurso. O cliente não tem ideia de que está se comunicando com um servidor. O proxy reverso cuida de capturar a solicitação do cliente e, em seguida, chama o recurso remoto em nome do cliente.
- No caso de uma arquitetura de microsserviços, o Spring Cloud Gateway recebe uma chamada de microsserviço de um cliente e a encaminha para o serviço upstream.

Configurando o Spring Cloud Gateway

- Para se comunicar com os serviços upstream, o gateway precisa saber como mapear a chamada recebida para a rota upstream. O Spring Cloud Gateway possui vários mecanismos para fazer isso, incluindo:
 - **Mapeamento automatizado de rotas usando descoberta de serviço:** O gateway pode utilizar mecanismos de descoberta de serviço, como o Eureka ou o Consul, para mapear automaticamente as rotas com base nas informações de registro dos serviços. Isso permite que o gateway acompanhe dinamicamente as alterações na infraestrutura de serviços.
 - **Mapeamento manual de rotas usando descoberta de serviço:** Além do mapeamento automatizado, o Spring Cloud Gateway também permite o mapeamento manual de rotas, onde você pode especificar explicitamente as rotas e seus destinos usando informações de descoberta de serviço.
- Esses mecanismos oferecem flexibilidade na configuração e no gerenciamento das rotas do gateway, permitindo que você escolha a abordagem mais adequada às necessidades do seu projeto.

Configurando roteamento automático de Rotas

- Quando a propriedade **`spring.cloud.gateway.discovery.locator.enabled`** está definida como **`true`**, o Spring Cloud Gateway é configurado para usar o DiscoveryClient (como Eureka, Consul ou outro servidor de descoberta suportado) para descobrir automaticamente os serviços registrados e criar rotas dinâmicas para eles.
- **Rotas dinâmicas:** Para cada serviço registrado no servidor de descoberta, o Gateway cria uma rota automaticamente. Isso elimina a necessidade de configurar manualmente as rotas para cada serviço no arquivo de configuração.
- **Segurança:** Com rotas dinâmicas habilitadas, o Gateway expõe automaticamente os serviços registrados. Recomenda-se adicionar mecanismos de autenticação e autorização.

Configurando roteamento automático de Rotas

- Para alterar o comportamento padrão das rotas geradas, use as propriedades:
 - *spring.cloud.gateway.discovery.locator.lower-case-service-id*: Define se os IDs dos serviços devem ser convertidos para letras minúsculas.
 - *spring.cloud.gateway.discovery.locator.include-expression*: Permite especificar quais serviços incluir nas rotas dinâmicas.
 - *spring.cloud.gateway.discovery.locator.exclude-expression*: Permite especificar quais serviços excluir das rotas dinâmicas.

Configurando roteamento automático de Rotas

- Exemplos:

```
1 server:
2   port: 8080
3 spring:
4   application:
5     name: APIGateway
6   cloud:
7     gateway:
8       discovery:
9         locator:
10           enabled: true
11           include-expression: "'user-service'.equals(serviceId) || 'order-service'.equals(serviceId)"
```

Apenas os serviços *user-service* e *order-service* terão rotas geradas.

Configurando roteamento automático de Rotas

- Exemplos:

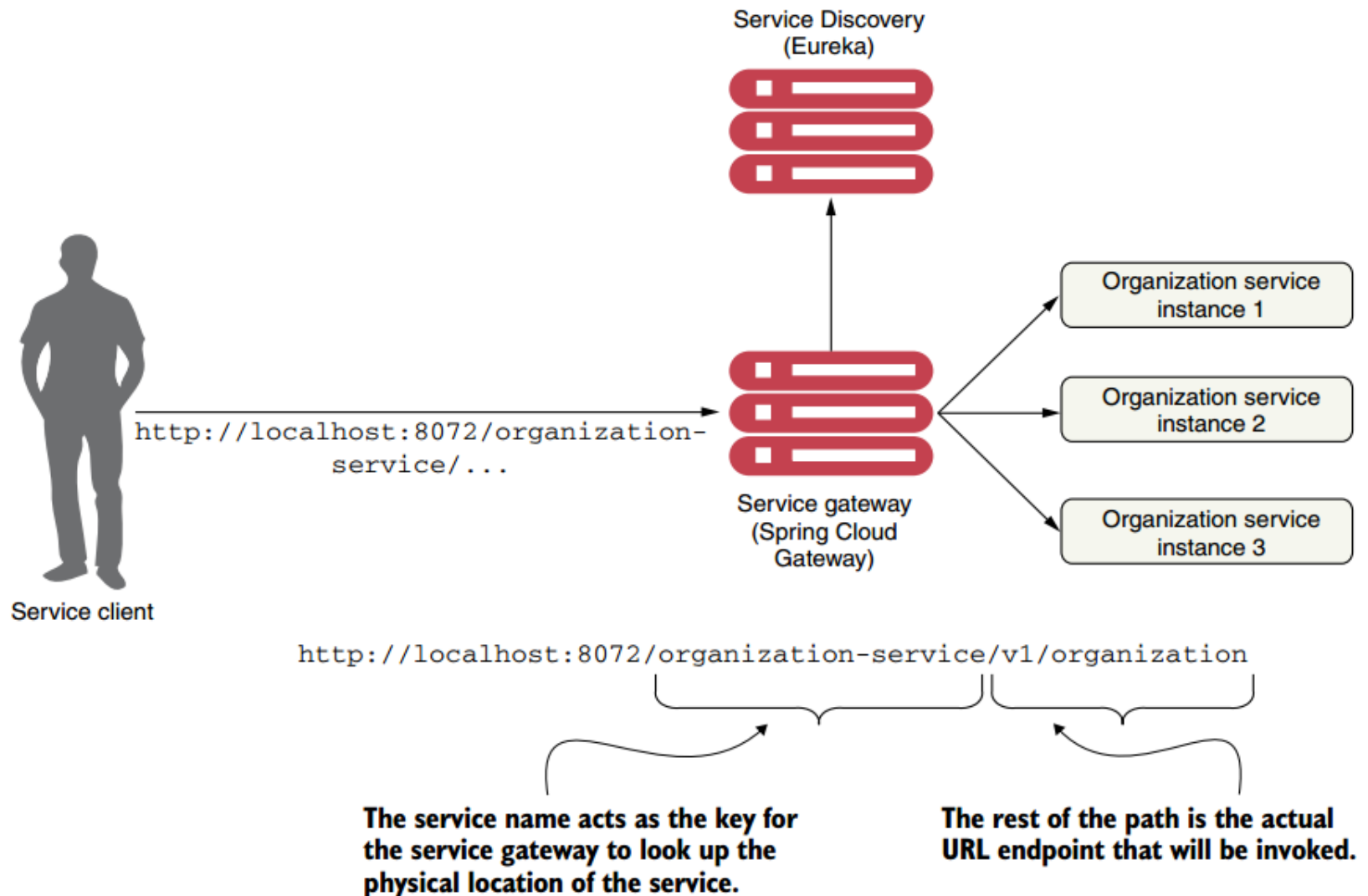
```
1 server:
2   port: 8080
3 spring:
4   application:
5     name: APIGateway
6   cloud:
7     gateway:
8       discovery:
9       locator:
10         enabled: true
11         include-expression: "serviceId.startsWith('app-')"
```

Serviços como app-user, app-order, e app-inventory serão incluídos.

Configurando roteamento automático de Rotas

```
1 spring:
2   application:
3     name: ServiceGatewayAPI
4   cloud:
5     loadbalancer:
6       ribbon:
7         enabled: false
8     gateway:
9       discovery:
10        locator:
11          enabled: true
12          lower-case-service-id: true
13
14 server:
15   port: 8080
16
17
18 eureka:
19   instance:
20     prefer-ip-address: true
21   client:
22     registerWithEureka: true
23     fetchRegistry: true
24     serviceUrl:
25       defaultZone: http://localhost:8761/eureka/
26
```

Funcionamento do roteamento automático de Rotas



Funcionamento do roteamento automático de Rotas

⏪ ⏩ ↻ 📖 ! localhost:8080/servicealpha/alpha

OK from 8091

⏪ ⏩ ↻ 📖 ! localhost:8080/servicebeta/beta

OK from 8091

⏪ ⏩ ↻ 📖 ! localhost:8080/servicebeta/alpha

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Sun Jan 23 10:06:37 BRT 2022

There was an unexpected error (type=Not Found, status=404).

Application	AMIs	Availability Zones	Status
SERVICEALPHA	n/a (1)	(1)	UP (1) - 10.0.0.167:servicealpha:8081
SERVICEBETA	n/a (1)	(1)	UP (1) - 10.0.0.167:servicebeta:8091
SERVICEGATEWAYAPI	n/a (1)	(1)	UP (1) - 10.0.0.167:ServiceGatewayAPI:8080

Funcionamento do roteamento automático de Rotas

- A vantagem de usar o Spring Cloud Gateway com o Eureka é que não só temos um único ponto de acesso para fazer chamadas, mas também podemos adicionar e remover instâncias de um serviço sem precisar modificar o gateway. Por exemplo, podemos adicionar um novo serviço ao Eureka, e o gateway roteará automaticamente as chamadas para ele porque está se comunicando com o Eureka sobre a localização dos pontos de extremidade físicos reais do serviço.
- Essa integração facilita a escalabilidade e a flexibilidade da arquitetura de microsserviços, permitindo que novas instâncias de serviços sejam adicionadas ou removidas sem interromper a funcionalidade do gateway. O gateway pode se adaptar dinamicamente às mudanças na infraestrutura, mantendo a comunicação eficiente entre o cliente e os serviços.

Observando as rotas expostas pelo Gateway

```
application.yml
1 spring:
2   application:
3     name: ServiceGatewayAPI
4   cloud:
5     loadbalancer:
6       ribbon:
7         enabled: false
8     gateway:
9       discovery:
10        locator:
11          enabled: true
12          lower-case-service-id: true
13
14  server:
15    port: 8080
16
17  eureka:
18    instance:
19      prefer-ip-address: true
20    client:
21      registerWithEureka: true
22      fetchRegistry: true
23      serviceUrl:
24        defaultZone: http://localhost:8761/eureka/
25
26  management:
27    endpoints:
28      web:
29        exposure:
30          include: "*"
31
```

Observando as rotas expostas pelo Gateway

GET http://localhost:8080/actuator/gateway/routes

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

KEY	VALUE	DESCRIPTION
Key	Value	Description

Body Cookies Headers (2) Test Results

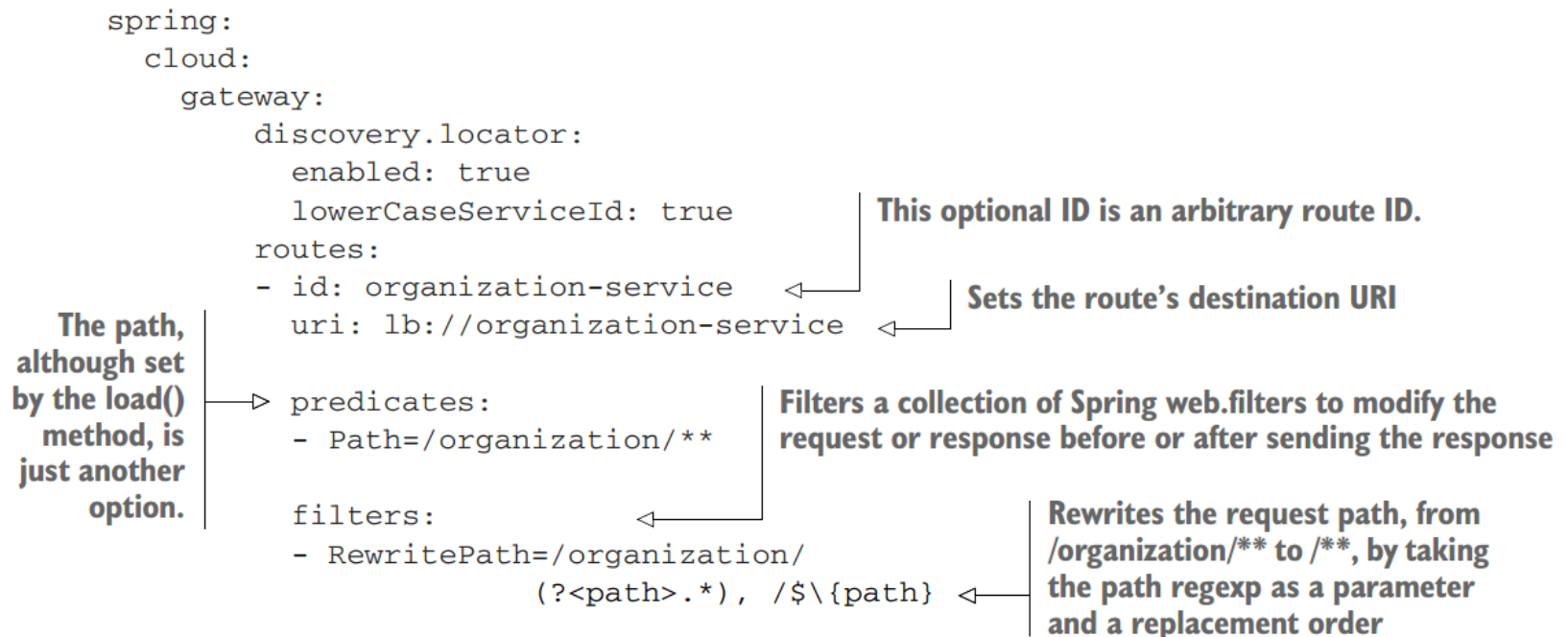
Status: 200 OK Time: 32 ms Size

Pretty Raw Preview Visualize JSON

```
1 {
2   {
3     "predicate": "Paths: [/servicealpha/**], match trailing slash: true",
4     "metadata": {
5       "jmx.port": "42313",
6       "management.port": "8081"
7     },
8     "route_id": "ReactiveCompositeDiscoveryClient_SERVICEALPHA",
9     "filters": [
10      "[[RewritePath /servicealpha/(?<remaining>.*) = '/${remaining}'], order = 1]"
11    ],
12     "uri": "lb://SERVICEALPHA",
13     "order": 0
14  },
15  {
16    "predicate": "Paths: [/servicegatewayapi/**], match trailing slash: true",
17    "metadata": {
18      "jmx.port": "35973",
19      "management.port": "8080"
20    },
21    "route_id": "ReactiveCompositeDiscoveryClient_SERVICEGATEWAYAPI",
22    "filters": [
23      "[[RewritePath /servicegatewayapi/(?<remaining>.*) = '/${remaining}'], order = 1]"
24    ],
25    "uri": "lb://SERVICEGATEWAYAPI",
26    "order": 0
27  },
28 }
```

Realizando o Mapeamento Manual das Rotas

- O Spring Cloud Gateway permite ser mais granular e definir explicitamente os mapeamentos de rotas, em vez de depender exclusivamente das rotas automatizadas criadas com o ID do serviço do Eureka.



Realizando o Mapeamento Manual das Rotas

- Uma rota escrita de forma manual consiste em um conjunto de condições (predicates) e um conjunto de ações (filters) a serem aplicadas.
- **predicates** são condições que determinam quando uma rota específica será ativada.
- Se todas as predicates de uma rota forem verdadeiras para uma solicitação, a rota será correspondida, e o Gateway a processará.

Realizando o Mapeamento Manual das Rotas

- Exemplos de Predicates:

```
1 server:
2   port: 8080
3 spring:
4   application:
5     name: APIGateway
6   cloud:
7     gateway:
8       routes:
9         - id: path_route
10           uri: lb://alphaservice
11           predicates:
12             - Path=/api/** # Correspondências para todos os caminhos começando com "/api/"
```

Realizando o Mapeamento Manual das Rotas

- Exemplos de Predicates:

```
1 server:
2   port: 8080
3 spring:
4   application:
5     name: APIGateway
6 cloud:
7   gateway:
8     routes:
9     - id: path_route
10      uri: lb://alphaservice
11      predicates:
12        - Method=GET # Apenas solicitações GET serão roteadas
```

Realizando o Mapeamento Manual das Rotas

- Exemplos de Predicates:

```
1 server:
2   port: 8080
3 spring:
4   application:
5     name: APIGateway
6   cloud:
7     gateway:
8       routes:
9         - id: path_route
10           uri: lb://alphaservice
11           predicates:
12             - Header=X-Request-Id, \d+ # Cabeçalhos "X-Request-Id" com valores numéricos
```

Realizando o Mapeamento Manual das Rotas

- Exemplos de Predicates:

```
1 server:
2   port: 8080
3 spring:
4   application:
5     name: APIGateway
6   cloud:
7     gateway:
8       routes:
9         - id: path_route
10           uri: lb://alphaservice
11           predicates:
12             - Path=/api/**
13             - Method=POST
14             - Header=Content-Type, application/json
```


Built-in Predicate Factories

Predicate	Description	Example
Before	Takes a date-time parameter and matches all the requests that happen before it.	Before=2020-03-11T...
After	Takes a date-time parameter and matches all the requests that happen after it.	After=2020-03-11T...
Between	Takes two date-time parameters and matches all the requests between them. The first date-time is inclusive, and the second one is exclusive.	Between=2020-03-11T..., 2020-04-11T...
Header	Receives two parameters, the name of the header, and a regular expression and then matches its value with the provided regular expression.	Header=X-Request-Id, \d+
Host	Receives an Ant-style pattern separated with the "." hostname pattern as a parameter. Then it matches the Host header with the given pattern.	Host=*.example.com
Method	Receives the HTTP method to match.	Method=GET
Path	Receives a Spring PathMatcher.	Path=/organization/{id}
Query	Receives two parameters, a required parameter and an optional regular expression, then matches these with the query parameters.	Query=id, 1

Realizando o Mapeamento Manual das Rotas

```
1 spring:
2   application:
3     name: ServiceGatewayAPI
4   cloud:
5     loadbalancer:
6       ribbon:
7         enabled: false
8   gateway:
9     routes:
10      - id: alpha-route
11        uri: lb://SERVICEALPHA/alpha
12        predicates:
13          - Path=/alpha/
14      - id: beta-route
15        uri: lb://SERVICEBETA/beta
16        predicates:
17          - Path=/beta/
```



localhost:8080/alpha/



localhost:8080/beta/

OK from 8091

: OK from 8091

Realizando o Mapeamento Manual das Rotas

GET http://localhost:8080/actuator/gateway/routes

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

KEY	VALUE	DESCRIPTION
Key	Value	Description

Body Cookies Headers (2) Test Results

Status: 200 OK Time: 27 ms

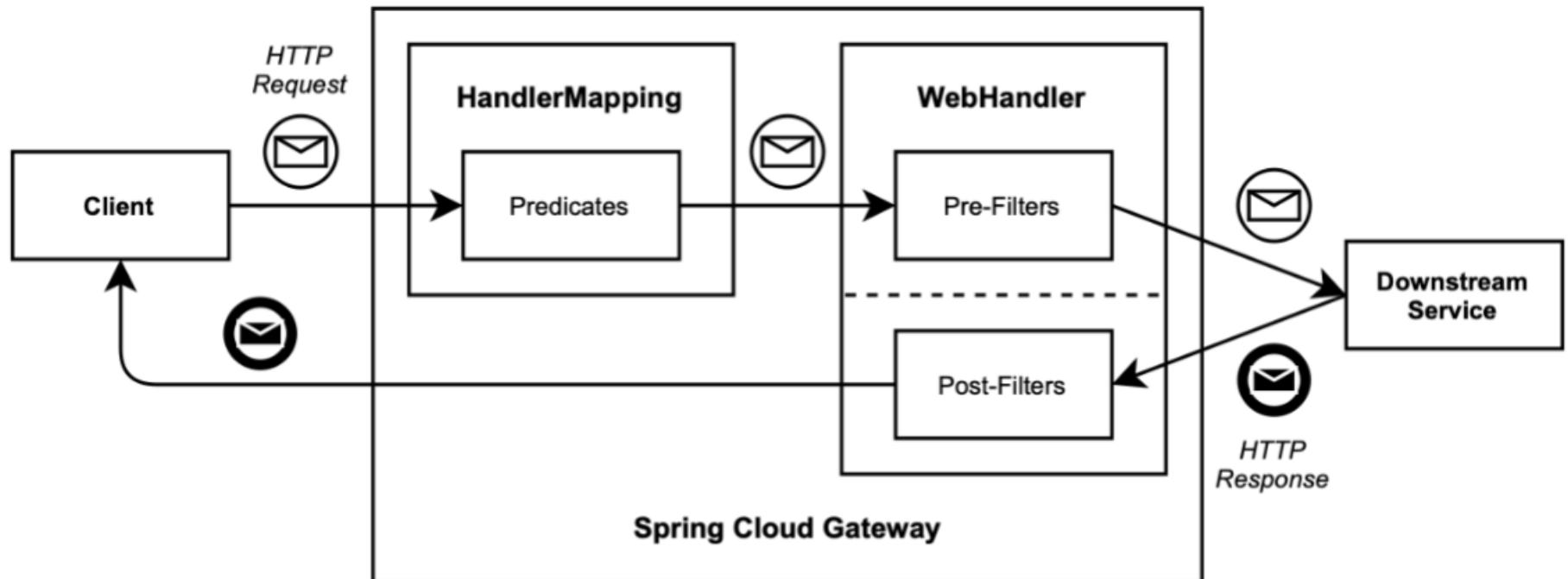
Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "predicate": "Paths: [/alpha/], match trailing slash: true",
4     "route_id": "alpha-route",
5     "filters": [],
6     "uri": "lb://SERVICEALPHA/alpha",
7     "order": 0
8   },
9   {
10    "predicate": "Paths: [/beta/], match trailing slash: true",
11    "route_id": "beta-route",
12    "filters": [],
13    "uri": "lb://SERVICEBETA/beta",
14    "order": 0
15  }
16 ]
```

Notice

- Quando utilizamos o mapeamento automático de rotas, em que o gateway expõe o serviço com base exclusivamente no ID do serviço do Eureka, se nenhuma instância do serviço estiver em execução, o gateway não exporá a rota para o serviço.
- No entanto, se mapearmos manualmente uma rota para um ID de descoberta de serviço e não houver instâncias registradas no Eureka, o gateway ainda mostrará a rota. Se tentarmos chamar a rota para o serviço inexistente, será retornado um erro HTTP 500.
- É importante considerar esses comportamentos ao configurar as rotas no gateway e garantir que o mapeamento corresponda à disponibilidade real dos serviços.

Como Realizar Filtragem no Spring Cloud Gateway



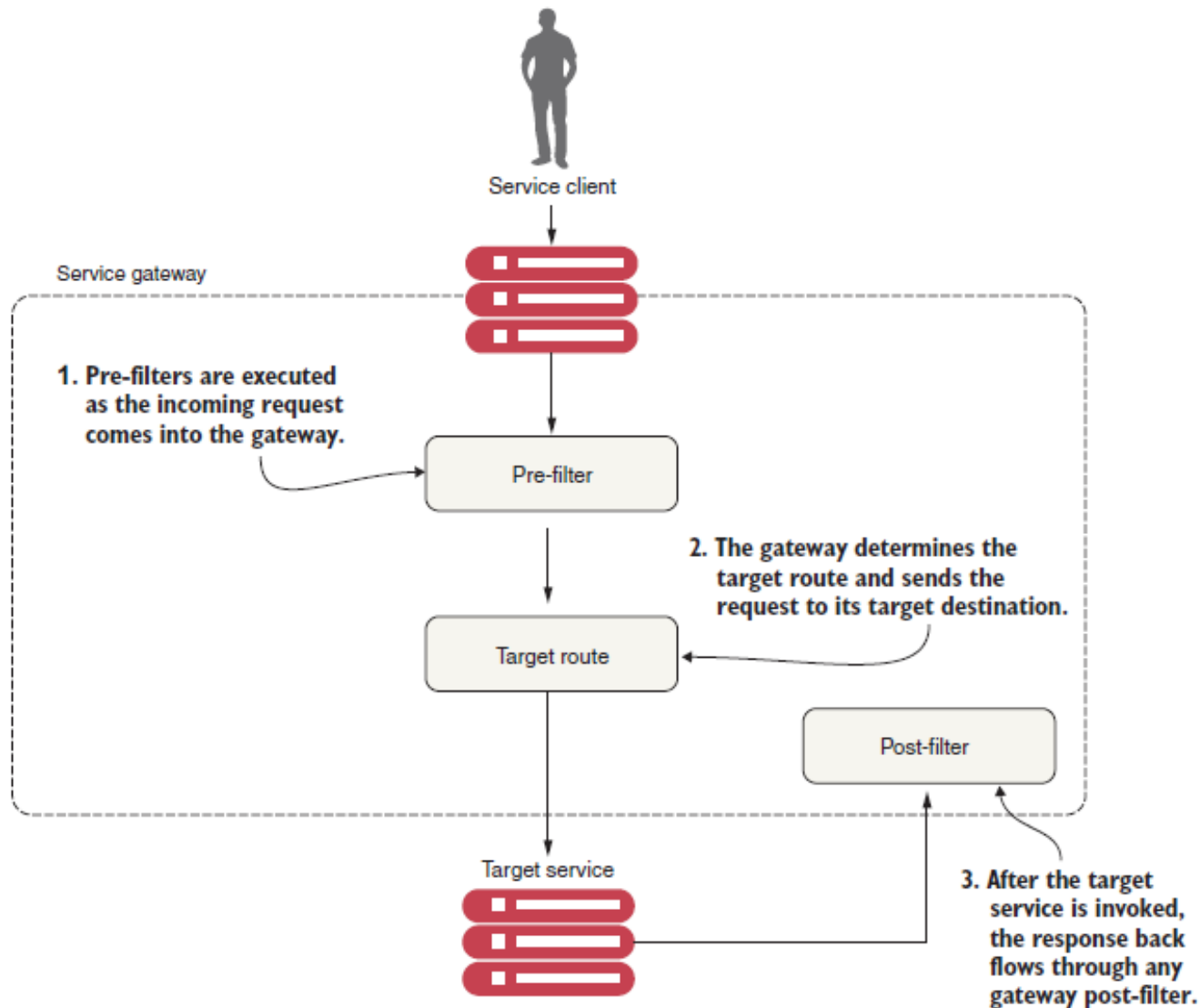
Built-in Filter Factories

- As Fábricas de Filtros incorporadas nos permitem injetar pontos de aplicação de políticas em nosso código e realizar uma ampla variedade de ações em todas as chamadas de serviço de maneira consistente.
- Em outras palavras, esses filtros nos permitem modificar as solicitações e respostas HTTP de entrada e saída. Eles oferecem a capacidade de interceptar as solicitações e respostas no gateway e aplicar transformações, validações, autenticação, autorização e outras políticas de segurança ou lógica de negócio.
- Os Filtros Factories podem ser usados para adicionar cabeçalhos, manipular o corpo da solicitação ou resposta, fazer log das chamadas, redirecionar solicitações, realizar autenticação baseada em tokens, entre outras ações.

Built-in Filter Factories

Predicate	Description	Example
AddRequestHeader	Adds an HTTP request header with the name and the value received as parameters.	AddRequestHeader= X-Organization-ID, F39s2
AddResponseHeader	Adds an HTTP response header with the name and the value received as parameters.	AddResponseHeader= X-Organization-ID, F39s2
AddRequestParameter	Adds an HTTP query parameter with the name and the value received as parameters.	AddRequestParameter= Organizationid, F39s2
PrefixPath	Adds a prefix to the HTTP request path.	PrefixPath=/api
RequestRateLimiter	Receives three parameters: replenishRate, which represents the requests per seconds that we want to allow the user to make; capacity, which defines how much bursting capacity is allowed; keyResolverName, which defines the name of a bean that implements the KeyResolver interface.	RequestRateLimiter= 10, 20, #{@userKeyResolver}
RedirectTo	Takes two parameters, a status and a URL. The status should be a 300 redirect HTTP code.	RedirectTo=302, http://localhost:8072

Built-in Filter Factories



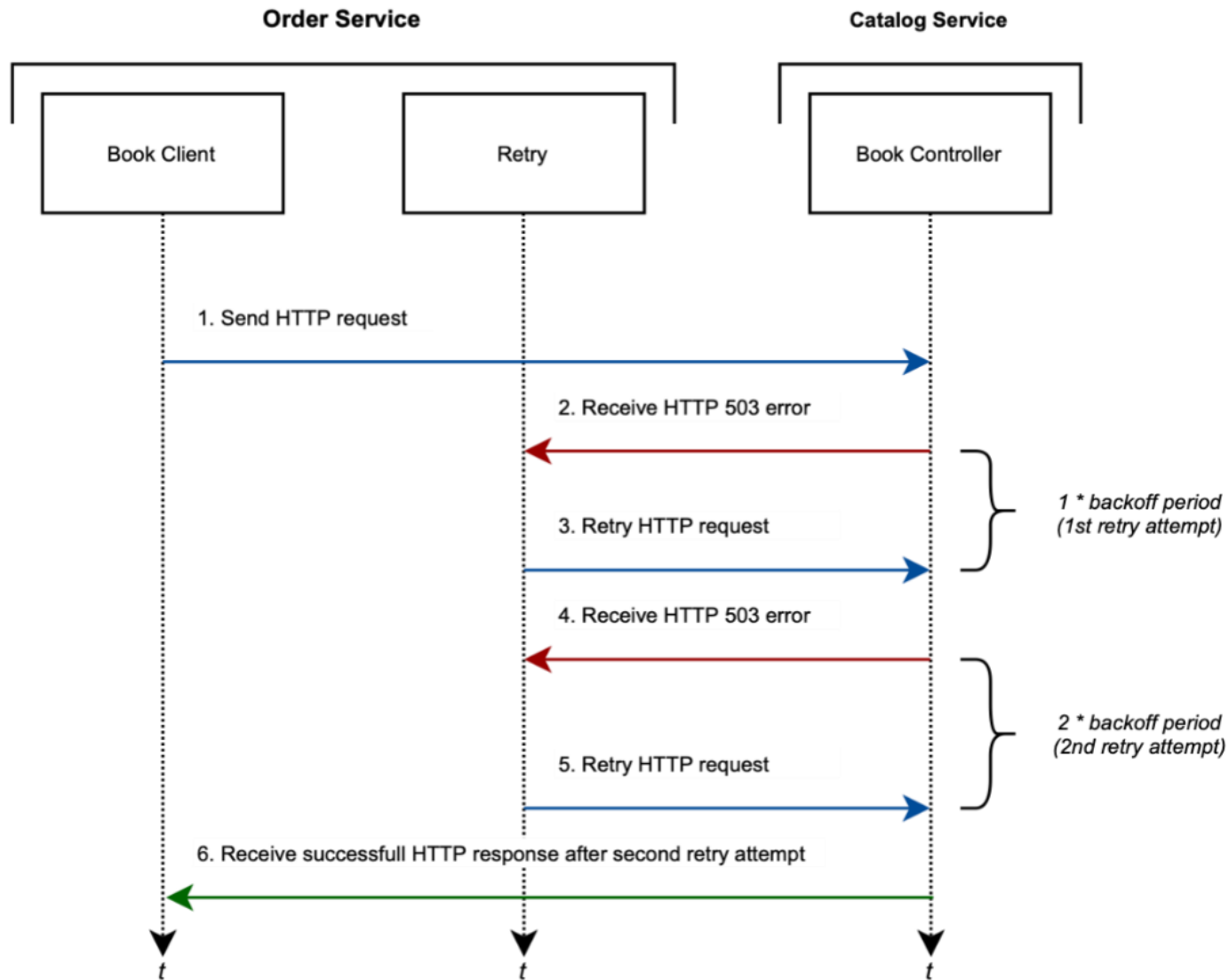
Usando Filtro de Retry

- O padrão de Retentativa (Retry) é útil quando um serviço de destino está momentaneamente indisponível.
- Quando um serviço não responde corretamente, é provável que esteja enfrentando alguns problemas e é improvável que ele se recupere imediatamente.
- Iniciar uma sequência de tentativas de retry uma após a outra pode tornar o sistema ainda mais instável.
- **Você não deseja lançar um ataque de negação de serviço (DoS) em suas próprias aplicações!**

Usando Filtro de Retry

- É importante aplicar uma estratégia adequada ao utilizar o padrão de Retentativa. Alguns pontos a considerar são:
 - Limitar o número de retentativas: Definir um limite razoável para o número de retentativas, a fim de evitar um loop infinito de tentativas sem sucesso.
 - Exponenciação de retentativas: Implementar uma estratégia de retentativa exponencial, onde o tempo entre as retentativas aumenta gradualmente. Isso evita sobrecarregar o serviço de destino com uma enxurrada de solicitações simultâneas.
 - Backoff aleatório: Introduzir um elemento aleatório no intervalo de tempo entre as retentativas. Isso evita que múltiplas instâncias do mesmo serviço cliente iniciem retentativas simultaneamente.

Usando Filtro de Retry



Quando devo usar Retry

- As solicitações idempotentes, como operações de leitura, podem ser repetidas sem causar problemas. Até mesmo algumas solicitações de gravação podem ser idempotentes. No entanto, não devemos repetir solicitações não idempotentes, pois corremos o risco de gerar estados inconsistentes.
- Quando configuramos repetições em um fluxo em que o usuário está envolvido, é importante equilibrar a resiliência e a experiência do usuário. Devemos considerar cuidadosamente quando e como repetir as solicitações, levando em conta o impacto na experiência do usuário. Em certos casos, pode ser necessário informar ao usuário que ocorreu um problema e fornecer opções alternativas.
- Além disso, não devemos repetir a solicitação se o serviço falhar com um erro recorrente. Por exemplo, se o serviço estiver completamente indisponível ou retornar um erro aceitável, como o erro 404, não faz sentido continuar repetindo a solicitação. Nesses casos, é importante lidar com a resposta de erro de forma apropriada e fornecer feedback ao usuário ou tomar ações alternativas.

Usando Filtro Retry

- **retries:** O número de tentativas de repetição que devem ser realizadas.
- **statuses:** Os códigos de status HTTP que devem ser repetidos, representados usando `org.springframework.http.HttpStatus`.
- **methods:** Os métodos HTTP que devem ser repetidos, representados usando `org.springframework.http.HttpMethod`.
- **series:** A série de códigos de status a serem repetidos, representados usando `org.springframework.http.HttpStatus.Series`.
- **exceptions:** Uma lista de exceções lançadas que devem ser repetidas.
- **backoff:** O intervalo de repetição exponencial configurado. As repetições são realizadas após um intervalo de espera de `firstBackoff * (fator ^ n)`, onde `n` é a iteração.
 - Se `maxBackoff` estiver configurado, o intervalo de espera máximo aplicado é limitado a `maxBackoff`.
 - Se `basedOnPreviousValue` for verdadeiro, o intervalo de espera é calculado usando `prevBackoff * fator`.

Usando Filtro Retry

```
spring:
  application:
    name: ServiceGatewayAPI
  cloud:
    loadbalancer:
      ribbon:
        enabled: false
  gateway:
    routes:
      - id: alpha-route
        uri: lb://SERVICEALPHA/alpha
        predicates:
          - Path=/alpha/
        filters:
          - name: Retry
            args:
              retries: 1000
              statuses: BAD_GATEWAY
              methods: GET,POST
              backoff:
                firstBackoff: 10ms
                maxBackoff: 50ms
                factor: 2
                basedOnPreviousValue: false

      - id: beta-route
        uri: lb://SERVICEBETA/beta
        predicates:
          - Path=/beta/
```

Usando Filtro Retry

```
cloud:
  gateway:
    default-filters:
      - name: Retry
        args:
          retries: 3
          methods: GET
          series: SERVER_ERROR
          exceptions: java.io.IOException
          backoff:
            firstBackoff: 50ms
            maxBackOff: 500ms
            factor: 2
            basedOnPreviousValue: false
```